

Code-Layout, Readability and Reusability

Date	06 July 2026
Team ID	
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the Credit Card Approval Prediction codebase in terms of its structure, readability, and reusability, based on a review of app.py, the preprocessing script (epic3), the model-development script (epic4), and the templates/static front-end files.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation used consistently across app.py, epic3, and epic4 scripts.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear separation: data/, model/, templates/, static/, and top-level epic scripts.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. MOB_THRESHOLD, DEPENDENTS_RATIO, FEATURE_COLUMNS, NOT_EMPLOYED_FLAG.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. build_feature_vector(), toggleEmployment() clearly state intent.
5	Code Comments	Inline and block comments explain logic	Yes	Strong docstrings in epic3/epic4 explain the vintage-analysis rationale and CV/threshold methodology; app.py explains the model-loading workaround.

S.No	Code Quality Parameter	Description	Followed	Remarks
6	Modular Design	Code is split into reusable functions/modules	Partial	app.py cleanly isolates build_feature_vector(); however epic3/epic4 are largely linear top-level scripts rather than importable functions/classes.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Category option lists (income types, occupations, etc.) and the education ordinal mapping are duplicated between epic3_p reprocessing_v3_vintage.py and app.py rather than shared from one source.
8	Error Handling	Exceptions and errors are handled gracefully	No	app.py has no try/except around model loading or prediction, and no server-side validation of form values beyond HTML constraints (e.g. a malformed request would raise an unhandled 500 error).

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	build_feature_vector()	Python / Flask	Called on every /predict request to convert form input into the model's expected feature vector	High
2	style.css	CSS	Shared styling across index.html and result.html	High
3	Category option lists (income/education/occupation/housing)	Python	Reused conceptually between the preprocessing pipeline (epic3) and the Flask app, though currently duplicated rather than imported from a shared module	Medium
4	Vintage-filtering + encoding pipeline	Python / pandas	Reusable for retraining the model whenever new application/credit records are added	Medium

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
5	Cross-validated tuning & OOF threshold selection routine	Python / scikit-learn	Reusable pattern for retuning the model without re-deriving methodology	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear, logical folder layout separating data, model, and web layers.
Readability	4	Descriptive naming and thorough explanatory comments, especially around modeling decisions.
Reusability	3	Core prediction logic is reusable, but shared constants/option lists are duplicated instead of centralized, and epic scripts use hardcoded local file paths (e.g. C:\Users\krish\Downloads\...).
Documentation / Comments	4	Docstrings explain not just what the code does but why (e.g. vintage-analysis justification, threshold tuning methodology).
Overall Score	4	Solid, maintainable codebase; main gaps are input/error handling and deduplicating shared constants into one module.